DOCKER · KODIGO BOOTCAMP · EJERCICIO PRÁCTICO

API de notas con persistencia

Dockerfile multi-stage + volúmenes: qué hice, por qué lo hice así y cómo explicarlo

Hugo Jovel · Kodigo Full Stack Jr 36 · Módulo 05 · Noviembre 2026

PUNTO DE PARTIDA

El reto y el resultado

Lo que pedía la actividad

- Dockerizar una mini API de notas.
- Usar un Dockerfile multi-stage.
- Demostrar que las notas siguen ahí aunque borres el contenedor.
- Tiempo: 45 minutos, en clase.

Lo que logré (con evidencia)

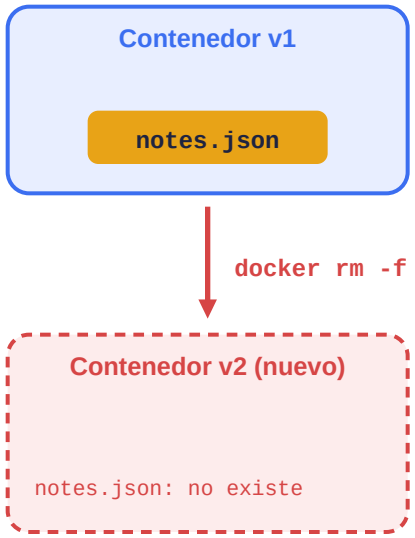
- Imagen api-notas:1.0 construida con multi-stage: 244 MB en disco, 62 MB de contenido.
- Con volumen: la nota conservó el mismo id (ff6740d5-...) después de docker rm -f.
- Sin volumen: al recrear el contenedor la lista quedó vacía: [ ].
- El contenedor reporta estado (healthy) gracias al HEALTHCHECK.

La idea en una frase: el contenedor es desechable; los datos importantes viven fuera.

EL CONCEPTO CLAVE

Contenedor desechable, datos duraderos

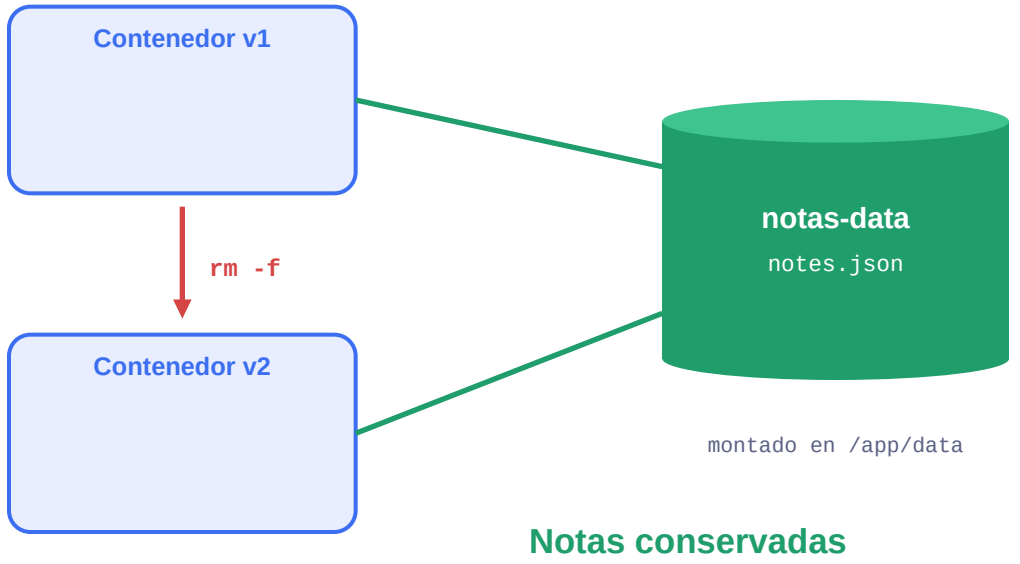
SIN volumen



Los datos vivían en la capa de escritura del contenedor. Al borrar el contenedor, esa capa se borra con él.

Notas perdidas [ ]

CON volumen



Notas conservadas

Analogía: el contenedor es una habitación de hotel (al hacer check-out la limpian y no queda nada tuyo). El volumen es la caja de seguridad: está fuera de la habitación y tus cosas siguen ahí.

EL PROCESO REAL

Cómo lo hice, paso a paso

1. **Verificar el entorno**
   which docker, docker context ls y un hello-world.
2. **Escribir la API**
   Node + Express; las notas se guardan en notes.json.
3. **Dockerfile multi-stage**
   Etapa deps (instala) y etapa runtime (imagen final).
4. **Construir la imagen**
   docker build -t api-notas:1.0 .
5. **Correr con volumen**
   -v notas-data:/app/data para que los datos vivan fuera.
6. **Probar y demostrar**
   Borrar el contenedor, recrearlo y comparar con el caso sin volumen.

Reparto de roles: yo y la IA

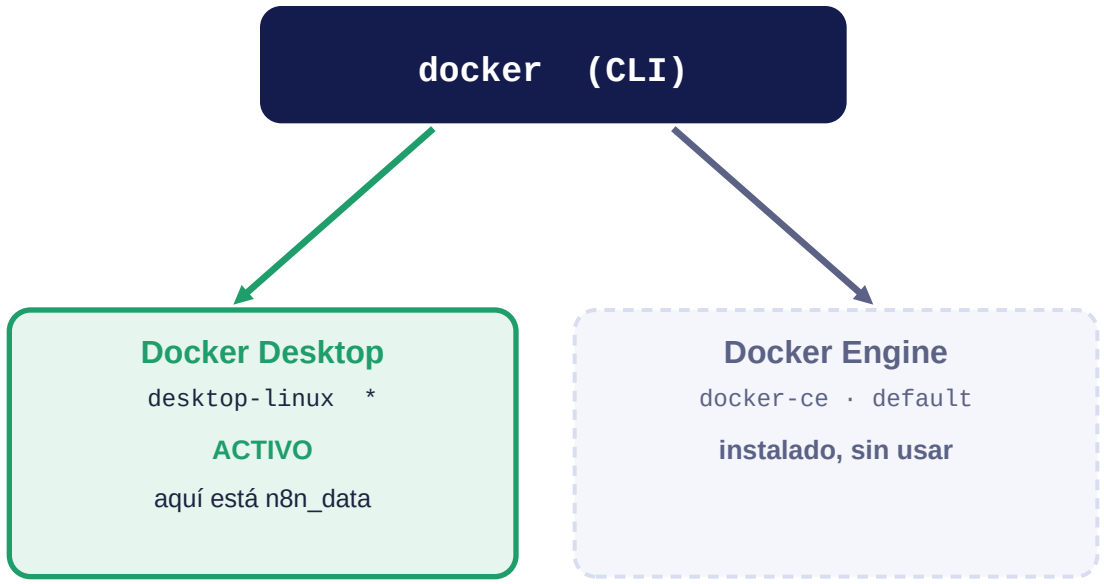
Mi parte: verificar mi propio entorno, ejecutar cada paso en mi terminal, interpretar los errores y comprobar los resultados con evidencia (ids, salidas, tamaños).

Parte de la IA (Claude): proponer la base del código y los comandos, y explicar el porqué de cada decisión técnica. La IA acelera; entender y validar lo que se ejecuta sigue siendo trabajo mío.

PASO 0

Verificar el entorno antes de empezar

*el contexto marcado con * decide a qué motor le hablas*



Cada motor tiene sus propias imágenes, contenedores y volúmenes. Saber cuál usas evita que tus datos parezcan "desaparecer".

Qué comprobé

- **`which docker · docker --version`**
  Docker 29.9 instalado en /usr/local/bin/docker
- **`docker context ls`**
  Contexto activo: desktop-linux (Docker Desktop)
- **`docker run --rm hello-world`**
  Descarga desde Docker Hub y ejecución correctas
- **`docker volume ls`**
  Aparece n8n_data, de las clases de n8n
- **`prueba mini con alpine + volumen`**
  El dato sobrevivió al contenedor: el principio de la actividad

EL CÓDIGO

La API de notas

MÉTODO	RUTA	QUÉ HACE
GET	`/health`	Estado del servicio (lo usa el healthcheck)
GET	`/notes`	Lista todas las notas
GET	`/notes/:id`	Devuelve una nota por id
POST	`/notes`	Crea nota: { title?, content }
DELETE	`/notes/:id`	Borra una nota

EJEMPLO REAL

```
$ curl -X POST localhost:3000/notes \
    -H 'Content-Type: application/json' \
    -d '{"title":"Primera","content":"Hola Docker"}'
{"id":"ff6740d5-9c1f-4e3a-837e-041eb27b8477",
 "title":"Primera","content":"Hola Docker", ...}
```

Decisiones de diseño

- Archivo JSON en /app/data: lo mínimo para demostrar persistencia en 45 minutos.
- Escritura atómica (archivo temporal + rename): un corte a mitad no corrompe el JSON.
- PORT y DATA_DIR por variables de entorno: la misma imagen sirve en cualquier lugar.
- Cierre ordenado con SIGTERM: docker stop responde rápido.
- Validaciones: content obligatorio, límites de tamaño, JSON inválido devuelve 400.
- Límite honesto: un solo proceso. Con varias réplicas usaría una base de datos.

Dockerfile multi-stage

```
# Etapa 1: dependencias
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# Etapa 2: imagen final
FROM node:22-alpine AS runtime
ENV NODE_ENV=production \
    PORT=3000 \
    DATA_DIR=/app/data
WORKDIR /app
COPY --from=deps --chown=node:node \
     /app/node_modules ./node_modules
COPY --chown=node:node package.json ./
COPY --chown=node:node src ./src
RUN mkdir -p /app/data && \
    chown node:node /app/data
USER node
EXPOSE 3000
HEALTHCHECK ...  # consulta /health
CMD ["node", "src/server.js"]
```

ETAPA 1 · deps

node:22-alpine + npm ci --omit=dev. Instala los paquetes de producción.

Se descarta: caché de npm y herramientas de instalación

`solo node_modules`

ETAPA 2 · runtime (imagen final)

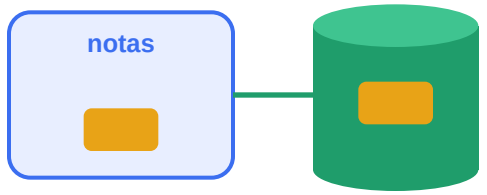
node_modules + src + package.json. Corre como usuario node, con HEALTHCHECK y carpeta de datos lista.

Honestidad técnica: aquí no hay compilación, así que la ganancia de tamaño es pequeña. Multi-stage es el patrón que escala: en TypeScript o Next.js, la etapa build deja fuera mucho peso.

LA DEMOSTRACIÓN EN CUATRO CUADROS

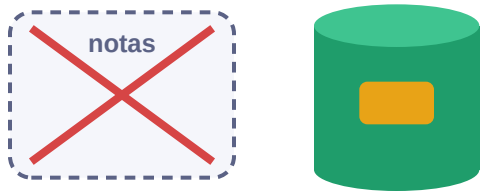
La prueba de persistencia

1 Crear una nota



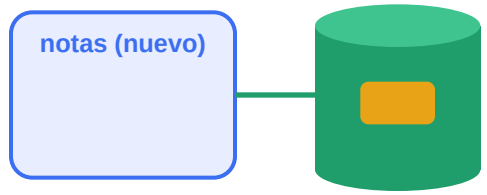

```
$ curl -X POST :3000/notes
    -d '{"content":...}'
id: ff6740d5-9c1f-...
```

2 Borrar el contenedor



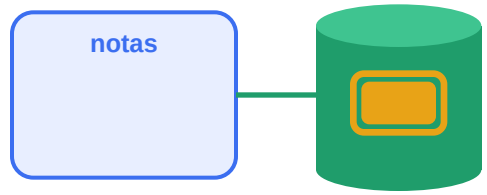

```
$ docker rm -f notas
$ docker ps -a
(vacío: ya no existe)
```

3 Contenedor nuevo, mismo volumen




```
$ docker run -d --name notas
   -p 3000:3000
   -v notas-data:/app/data
   api-notas:1.0
```

4 La nota sigue ahí




```
$ curl localhost:3000/notes
id: ff6740d5-9c1f-...
mismo id = mismo dato
```

CONTRASTE: la misma prueba SIN volumen

`1 crear nota` → `2 docker rm -f` → `3 docker run (sin -v)` → `4 GET /notes = [ ]`

MI TERMINAL, SIN RETOQUES

Evidencia real

1 · CON VOLUMEN: LA NOTA SOBREVIVE

```
$ docker rm -f notas
$ docker ps -a            # vacío
$ docker run -d --name notas -p 3000:3000 \
    -v notas-data:/app/data api-notas:1.0
$ curl localhost:3000/notes
[{"id":"ff6740d5-9c1f-4e3a-837e-041eb27b8477",
  "title":"Primera","content":"Hola Docker", ...}]
```

2 · SIN VOLUMEN: SE PIERDE

```
$ docker run -d --name notas-sin-vol -p 3001:3000 api-notas:1.0
$ curl -X POST localhost:3001/notes ... {"content":"efímera"}
[{"id":"82a109b4-7765-4db3-af94-ce3e44151a55", ...}]
$ docker rm -f notas-sin-vol
$ docker run -d --name notas-sin-vol -p 3001:3000 api-notas:1.0
$ curl localhost:3001/notes
[]
```

3 · EL ARCHIVO DENTRO DEL VOLUMEN

```
$ docker run --rm -v notas-data:/data alpine \
    cat /data/notes.json
[
  {
    "id": "ff6740d5-9c1f-4e3a-837e-041eb27b8477",
    "title": "Primera",
    "content": "Hola Docker",
    "createdAt": "2026-11-04T01:56:31.281Z"
  }
]
```

4 · La imagen

244 MB

en disco · 62 MB de contenido · api-notas:1.0

- Contenedor en estado (healthy)
- Mayor parte del peso: la imagen base de Node, no el código

LAS DECISIONES

¿Por qué así y no de otra forma?

DECISIÓN	LO QUE ELEGÍ	ALTERNATIVA	POR QUÉ
Imagen	**multi-stage**	una sola etapa	Es lo que pide la actividad y el patrón que escala. Aquí la ganancia es pequeña (no hay compilación).
Datos	**volumen nombrado**	bind mount (carpeta del host)	Lo administra Docker, es portable y hereda permisos. El bind mount depende de rutas y permisos del equipo.
Usuario	**node (no-root)**	root (por defecto)	Si alguien vulnera la app, tiene menos poder dentro del contenedor.
Dependencias	**npm ci --omit=dev**	npm install	Instala exactamente lo del package-lock (reproducible) y sin paquetes de desarrollo.
Imagen base	**node:22-alpine**	node:22-slim (Debian)	Más liviana. slim es la opción si algún paquete nativo no compila en Alpine.
Almacén	**archivo JSON**	SQLite o PostgreSQL	Mínimo para demostrar persistencia en 45 min. Con concurrencia o réplicas, base de datos.
Instrucción VOLUME	**no la usé**	VOLUME ["/app/data"]	Crea volúmenes anónimos y taparía el contraste: sin -v los datos deberían perderse.

PARA LLEVARTE

Buenas prácticas y consejos

Verifica tu entorno primero
docker context ls y hello-world antes de escribir una línea.

Agrega HEALTHCHECK
docker ps te muestra (healthy) y detectas caídas rápido.

Escribe un .dockerignore
Deja fuera node_modules, data y .git: build rápido y sin basura.

Maneja SIGTERM
Así docker stop cierra al instante en vez de esperar 10 segundos.

Fija la versión de la imagen base
node:22-alpine, no :latest: builds repetibles.

Datos importantes, fuera del contenedor
Un volumen para todo lo que no puedas perder.

Copia package*.json antes del código
Aprovechas la caché de capas: reinstala solo si cambian dependencias.

Cuidado al limpiar
docker volume ls antes de borrar. Nunca prune --volumes a ciegas.

No corras como root
USER node reduce el daño si la app se compromete.

Espera a que arranque
Usa sleep o el healthcheck antes de probar con curl.

LO QUE SALIÓ MAL (Y ESTÁ BIEN CONTARLO)

Errores reales y qué enseñaron

curl: conexión reinicializada

QUÉ PASÓ

Lancé el curl en el mismo instante en que arrancó el contenedor. Docker ya había abierto el puerto, pero Node aún no escuchaba.

LECCIÓN

Esperar (sleep 2) o mirar docker ps hasta ver (healthy).

Dos motores de Docker

QUÉ PASÓ

Tenía instalados Docker Desktop y docker-ce a la vez. Cada uno guarda sus propias imágenes y volúmenes.

LECCIÓN

Correr docker context ls y saber cuál está activo antes de buscar tus datos.

El prompt pegado a la respuesta

QUÉ PASÓ

La API devuelve JSON sin salto de línea final, y el prompt de la terminal quedó pegado al resultado.

LECCIÓN

Agregar ; echo al curl, o usar una herramienta como jq para leer mejor.

Un error documentado enseña más que uno escondido. Contarlos demuestra que lo ejecuté yo.

PREGUNTA CLAVE 1

¿Por qué Debian y no Windows?

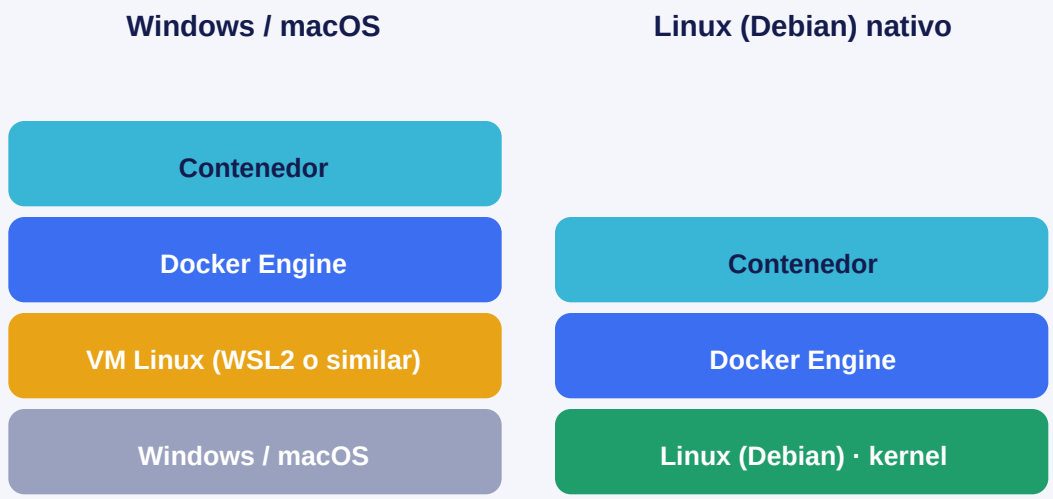
1. **Los contenedores son tecnología de Linux**
   Comparten el kernel de Linux. En Windows o macOS, Docker necesita una máquina virtual Linux debajo.

2. **Se despliega en servidores Linux**
   Mismo sistema en desarrollo y producción: menos sorpresas de "en mi máquina sí funciona".

3. **Las herramientas son nativas**
   bash, curl, ssh, git y grep funcionan igual en mi equipo, en un servidor y en CI.

4. **Ya venía trabajando en Debian**
   Mis clases de n8n y de Docker ya estaban ahí: un solo entorno, sin cambiar de contexto.

5. **Liviano, estable y gratuito**
   Menos consumo de recursos y sin licencias.



Mi caso real: Debian + Docker Desktop, que también usa su propia VM Linux. Aun así, el sistema, los comandos y los scripts son los de un servidor. Y con WSL2 Windows también sirve: la clave es trabajar en Linux.

Mi razón personal (completar antes de exponer):

_____

PREGUNTA CLAVE 2

¿Por qué terminal y no una interfaz?

1. **La terminal es la interfaz original**
   Docker Desktop y las extensiones de VS Code ejecutan estos mismos comandos por debajo.

2. **Es reproducible**
   Copio los comandos al README y cualquiera los repite igual. Los clics no se documentan.

3. **Funciona en cualquier servidor**
   Por SSH no hay interfaz gráfica: la terminal es lo único que hay.

4. **Los mismos comandos van a scripts y CI/CD**
   Lo que aprendo hoy lo automatizo mañana, por ejemplo en GitHub Actions.

5. **Veo lo que realmente pasa**
   Los errores y los logs aparecen completos: el connection reset, el estado healthy.

Cuándo uso cada herramienta

Terminal

Construir, correr y probar: docker build, run, logs, curl.

Docker Desktop

Observar: ver contenedores, volúmenes y uso de disco (como la captura de n8n_data).

Editor (VS Code)

Escribir el código y el Dockerfile. Tiene terminal integrada.

No es uno u otro: terminal para ejecutar, interfaz para observar, editor para escribir.

PASAR EL APRENDIZAJE

Cómo enseñarlo a otros

1 · Una analogía

Contenedor = habitación de hotel: al hacer check-out la limpian y no queda nada tuyo. Volumen = caja de seguridad: está fuera de la habitación y tus cosas siguen ahí cuando entras a otra.

2 · Demo en vivo

Muestra primero el fallo (sin volumen) y después la solución (con volumen). Cinco minutos y tres comandos clave: run, rm -f, run. Mismo id = misma nota.

3 · Que lo rompan

Antes de ejecutar, pregunta: ¿qué va a pasar con la nota? Deja que predigan. Luego pídeles cambiar el nombre del volumen y ver qué pasa: aparece vacío.

Consejos para enseñar bien

- Empieza por el problema, no por el comando.
- Un concepto a la vez: primero volúmenes, luego multi-stage.
- Entrega el README con comandos copiables.
- Enseña también tus errores: dan confianza y enseñan más.

PARA EL DÍA DE LA EXPOSICIÓN

Guion de 7 minutos

0:00 – 1:00	**El problema** Los contenedores son desechables. ¿Dónde quedan mis datos?
1:00 – 2:00	**Mi entorno** Terminal en Debian, Docker Desktop, contexto verificado.
2:00 – 3:30	**API + Dockerfile** Mostrar las dos etapas y qué pasa de una a otra.
3:30 – 5:30	**Demo en vivo** Nota → rm -f → run → misma nota. Luego el contraste sin volumen.
5:30 – 6:30	**Decisiones** No-root, .dockerignore, healthcheck, volumen nombrado.
6:30 – 7:00	**Cierre** Tres ideas y preguntas.

Antes de exponer

- [x] docker ps limpio (sin notas ni notas-sin-vol)
- [x] Puertos 3000 y 3001 libres
- [x] Imagen api-notas:1.0 ya construida
- [x] Volumen notas-data vacío para partir de cero
- [x] Terminal con letra grande
- [x] Capturas de evidencia abiertas

Plan B: si falla la demo en vivo, muestra la página de evidencia real. Tenerla lista vale más que improvisar.

PREPÁRATE

Preguntas que te pueden hacer

¿Qué es un volumen?

Un espacio de almacenamiento administrado por Docker, fuera del ciclo de vida del contenedor.

¿Por qué no usas una base de datos?

Para demostrar persistencia en 45 minutos basta un archivo. Con réplicas o concurrencia, usaría PostgreSQL.

¿Qué pasa si borro el volumen?

Se pierden las notas. docker volume rm es destructivo; el contenedor, no.

¿Dónde está físicamente el volumen?

Dentro de la VM de Docker Desktop. Para verlo: docker run --rm -v notas-data:/data alpine cat /data/notes.json.

¿Por qué multi-stage si no compilas nada?

Es el patrón que escala. Aquí la ganancia es pequeña; con TypeScript o Next.js sería grande.

¿Y docker compose?

Es el siguiente paso: levantar API y base de datos juntas con un solo archivo y un solo comando.

RESUMEN

Tres ideas para llevarse

1. **Contenedor desechable, datos duraderos**
   Lo que importa vive en un volumen, no en el contenedor.

2. **Verifica antes de construir**
   Conocer tu entorno (qué motor, qué contexto) evita horas de confusión.

3. **Cada decisión tiene un porqué**
   No-root, multi-stage, volumen nombrado: poder explicarlo es lo que te hace dev.

Siguientes pasos

- docker compose: API + PostgreSQL juntos.
- Publicar la imagen en Docker Hub.
- GitHub Actions para construir la imagen en cada push.
- Pasar la API a TypeScript y ver el multi-stage con compilación real.

Contenedor desechable, datos duraderos.